RESEARCH PROTOTYPE · PROJECT EXPLAINER

Hallucination Detection Using Machine Learning and Evidence Verification

How the system works, how it is built, the datasets and experiments, and the results measured so far.

Prepared for	Sreejay
Document date	22 October 2026
Code location	`Desktop\hallucination-detection`
Status	Pipeline working end to end; Experiment A and the RAGTruth baselines measured; fine-tuning (B), RAGTruth with the fine-tuned model, and the controlled test (D) still to run.

Contents

1. Summary
2. The problem and the approach
3. Architecture
4. How each stage works
5. The dashboard and command line
6. Datasets
7. Experiments and results
8. Limitations
9. How to run it
10. Code map and testing

Every number in this document comes from a run on Sreejay's computer on 22 October 2026, saved as a JSON file in `results\`. Unit-test fixtures and mock runs are never reported as results.

1. Summary

Large language models sometimes state false things fluently ("hallucinations"). This project builds a system that **checks an LLM's answer claim by claim against external evidence**, rather than trusting the model. It does not build a new LLM.

- A user asks a question. An LLM (by default `openai/gpt-oss-20b` via the Groq API) writes an answer.
- The answer is split into candidate claims (sentences) with spaCy.
- For each claim, evidence passages are retrieved from Wikipedia.
- A separate natural language inference (NLI) classifier judges each (evidence, claim) pair and labels the claim **Supported**, **Contradicted** or **Insufficient Evidence**. If no verdict can be made, the label is **Retrieval Failure** or **Verification Error**.
- A Streamlit dashboard shows the answer, every claim, its evidence with links, the model scores and the verdict.

> **Results so far.** On 2,950 held-out FEVER dev pairs, the pretrained `cross-encoder/nli-deberta-v3-base` verifier reaches **80.2% accuracy, 0.797 macro F1**, or 80.9% / 0.807 with thresholds tuned on validation (Experiment A). On RAGTruth, where the task is different and harder, it detects hallucinated sentences only modestly better than chance (strict F1 0.16). Fine-tuning on FEVER (Experiment B) is the next step.

2. The problem and the approach

Why hallucinations are hard to catch

An LLM produces text that is likely given its training, not text that has been checked. Its confidence and fluency look the same whether a statement is right or wrong, so the model's own output cannot be used to judge itself reliably.

Approach: evidence-based verification

The project treats detection as **fact verification**: break the answer into checkable statements, find independent evidence, and use a classifier trained for textual entailment to decide whether the evidence supports, contradicts, or does not settle each statement. This follows the FEVER fact-verification task (Thorne et al., 2018).

Design principles

- **Separation.** The generator and the verifier are different models. The verifier sees only the claim text and the evidence, never the question or the generator's reasoning.
- **Absence is not falsity.** If no evidence is found, the claim is Insufficient Evidence, never Contradicted.
- **Honest outputs.** Scores are shown as uncalibrated model scores, not probabilities of truth. Network or model failures get their own labels instead of a guessed verdict.
- **Untrusted evidence.** Retrieved text is shown verbatim and never followed as instructions.

- **Reproducibility.** Seeds, sample sizes, model names and settings are written into every results file.

3. Architecture

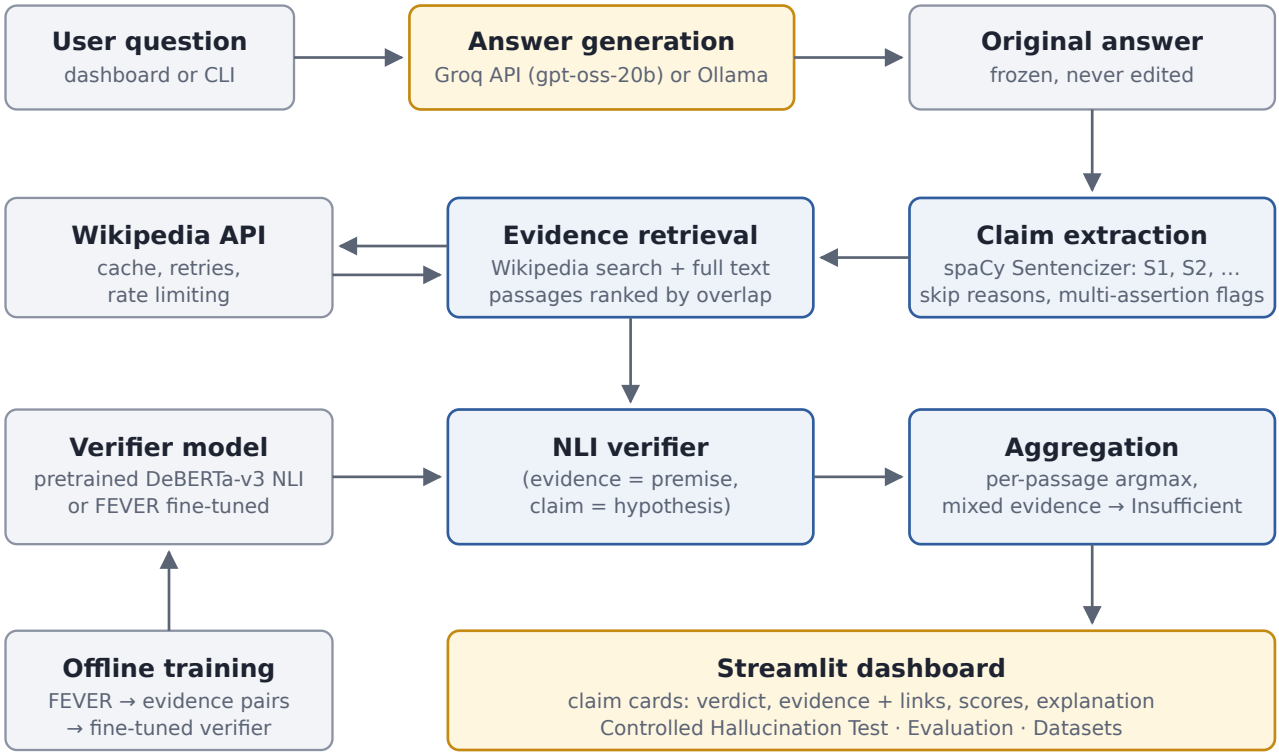


Figure 1. Runtime pipeline (blue: verification system; amber: generation and presentation; grey: inputs, data and offline training).

The system has two halves that never share state:

- **Generation** (amber): any LLM behind one `LLMClient` interface. The provider is a setting (`LLM_PROVIDER`): Groq, OpenRouter, OpenAI, any OpenAI-compatible endpoint, or local Ollama.
- **Verification** (blue): extraction, retrieval and NLI classification. The verifier is either a pretrained NLI cross-encoder or a checkpoint fine-tuned on FEVER by the scripts in `training/`.

Each stage is a separate Python module with its own tests, so any one of them can be swapped or evaluated on its own.

4. How each stage works

4.1 Answer generation

- Default: Groq's free API with `openai/gpt-oss-20b`, an open-weight 20B model. The original plan used Llama 3.2 3B. Groq lists no Llama chat models for this account, so gpt-oss-20b is used instead; Llama is still available through OpenRouter or local Ollama.
- The API key lives only in `.env` (ignored by git) and never appears in the UI, logs or error messages.
- Errors are typed (bad key, rate limit, timeout, unknown model, malformed reply) and shown plainly. Rate limits are retried using the server's `Retry-After` hint. Nothing is silently substituted.

- Each answer records which provider and model produced it.

4.2 Claim extraction

- spaCy's rule-based Sentencizer splits the answer line by line. Every sentence gets an id (S1, S2, …) and its exact character span.
- Questions, greetings, opinions, advice and fragments under four words are shown but not verified, with the reason.
- A sentence is not always one fact. Sentences with signs of several assertions (semicolons, coordinated or relative clauses, parentheses, two or more numbers) are **flagged** and verified as a whole, so they are Supported only if everything in them is supported.
- Optional: the LLM can decompose sentences into atomic sub-claims (S2.1, S2.2). Sub-claims that cannot be traced back to the answer are dropped with a warning.

4.3 Evidence retrieval

1. The MediaWiki search API finds the top pages for the claim, with a keyword fallback.
2. The full plain text of each article and its URL are fetched.
3. Articles are cut into passages, ranked by lexical overlap with the claim (BM25-style), and the top 3 passages are passed on.
4. Timeouts, retries with backoff, request spacing and a disk cache keep it polite and repeatable.

4.4 Verification

Each (passage, claim) pair goes through the NLI cross-encoder. The page title is prefixed to the passage so pronouns can be resolved.

NLI class / FEVER class	App label	Meaning
entailment / SUPPORTS	Supported	The evidence states what the claim says.
contradiction / REFUTES	Contradicted	The evidence states something incompatible with the claim.
neutral / NOT ENOUGH INFO	Insufficient Evidence	The evidence does not settle the claim. This does not mean false.

- **Per passage:** the highest-scoring class. Minimum-score thresholds are off unless tuned on validation data and chosen explicitly.
- **Across passages:** if some passages support the claim and others contradict it, the result is Insufficient Evidence marked "mixed evidence". Otherwise Supported beats Contradicted beats Insufficient.
- **Fallback:** if no NLI model is installed, an LLM-judgment verifier can be used. It is labelled "not a trained classifier" everywhere it appears.

5. The dashboard and command line

Dashboard (`streamlit run app.py`, four tabs):

- **Overview:** live status of the generation API, Wikipedia and the verifier in use.
- **Ask & verify:** question box, the unmodified answer, and a card per claim with the verdict, retrieved passages (title, link, text), per-passage scores and the explanation. Results download as JSON.
- **Controlled Hallucination Test:** a separate copy of the answer gets one known-false claim, either one of 8 hand-reviewed false statements or a sentence the user edits. Both versions go through the same pipeline and are shown side by side, with the injected claim highlighted and the actual verdict reported, including misses. The original answer is never modified.
- **Evaluation:** runs and displays saved FEVER, RAGTruth and controlled-test results; compares baseline and fine-tuned models.
- **Datasets:** file status and dataset statistics.

Command line (`python -m src.cli …`): `status`, `ask`, `download-nli`, `eval-fever`, `eval-ragtruth`, `controlled-test`, `mutate`, `dataset-report`. Training is in `training/prepare_fever.py`, `train_verifier.py` and `evaluate_verifier.py`.

6. Datasets

6.1 FEVER (primary: training and evaluating the verifier)

FEVER (Fact Extraction and VERification) contains claims written from Wikipedia, each labelled SUPPORTS, REFUTES or NOT ENOUGH INFO, with evidence given as (page, sentence number) in a 2017 Wikipedia dump.

File	Claims	SUPPORTS	REFUTES	NEI
train.jsonl	145,449	80,035	29,775	35,639
shared_task_dev.jsonl (held out)	19,998	6,666	6,666	6,666

`prepare_fever` turns these into (evidence text, claim, label) pairs using the wiki dump:

- A validation slice is held out from train, grouped by claim text so identical claims never cross the boundary. It is used only for model selection and threshold tuning.
- The official dev file is used only for final evaluation; the 2 dev claims that also appear in train are removed.
- Subsets are class-balanced and seeded (seed 13) so runs fit on a CPU.
- SUPPORTS/REFUTES evidence = the annotated sentences. NOT ENOUGH INFO has no annotated evidence, so sentences are sampled from the page whose title best matches the claim (nearest-page heuristic).

Prepared split	Pairs	SUPPORTS	REFUTES	NEI	Dropped
Train	8,888	2,972	2,977	2,939	112
Validation	2,959	995	993	971	41
shared_task_dev (test)	2,950	988	989	973	50

6.2 RAGTruth (supplementary: a different task)

RAGTruth (Niu et al., 2024) contains 17,790 responses from six LLMs (GPT-4, GPT-3.5, Mistral-7B and three Llama-2 sizes) to QA, summarisation and data-to-text prompts. Annotators marked the exact character spans that conflict with, or are not supported by, the **provided source document**. It is never mixed with FEVER and never used for training.

- Span types: Evident Baseless Info 6,237 · Evident Conflict 5,324 · Subtle Baseless Info 2,527 · Subtle Conflict 201.
- Splits: 15,090 train, 2,700 test. Evaluation uses the test split, QA and Summary tasks only (Data2txt contexts are JSON tables).
- **Conversion to sentence labels:** the response is split with the same Sentencizer; a sentence is hallucinated if any counted span overlaps it (spans marked `implicit_true` are excluded). A response is hallucinated if it has any counted span. The evidence is the response's own source document, not Wikipedia.
- **Strict** prediction: the sentence was Contradicted. **Lenient**: the sentence was anything but Supported.

7. Experiments and results

Experiment	Question	Status
A. Pretrained baseline on FEVER	How well does an off-the-shelf NLI model verify FEVER claims given gold evidence?	DONE
B. Fine-tuned verifier on FEVER	Does fine-tuning DeBERTa-v3-base on FEVER pairs beat A on the same held-out split?	PENDING
C. RAGTruth transfer	Does the verifier detect real LLM hallucinations against a source document?	BASELINES DONE FINE-TUNED PENDING
D. Controlled hallucination test	When a known-false claim is inserted into a real answer, is it caught while the rest is unchanged?	PENDING

7.1 Experiment A: pretrained NLI baseline on FEVER

Model: `cross-encoder/nli-deberta-v3-base`, used as downloaded (not trained by this project). Input: gold evidence and claim.

Split	Decision rule	n	Accuracy	Macro F1
Validation (from train)	thresholds tuned here (support ≥ 0.8, contradiction ≥ 0.9)	2,959	81.1%	0.808
shared_task_dev (held out)	argmax, no thresholds	2,950	80.2%	0.797
shared_task_dev (held out)	validation-tuned thresholds applied	2,950	**80.9%**	**0.807**

- Validation and held-out scores are close, so the result is not an artefact of tuning on validation.
- Thresholds tuned on validation transfer to held-out data: +0.7 points accuracy and +0.010 macro F1 (about 20 more pairs correct out of 2,950). The gain is small, so it is reported as a modest improvement.
- About a third of the pairs are NOT ENOUGH INFO with nearest-page evidence, which may make that class easier than in real use.

7.2 Experiment C: RAGTruth with pretrained verifiers

Same 300 test responses for both models (QA + Summary, seed 13): 1,665 sentences, of which **59 are hallucinated (3.5%)**; 48 of 300 responses (16%) are hallucinated. 167 sentences skipped by the extractor were excluded.

Level / rule	Model	Precision	Recall	F1	Flags raised
Sentence, strict	nli-deberta-v3-xsmall	9.0%	28.8%	0.138	188
	nli-deberta-v3-base	12.6%	22.0%	0.160	103
Sentence, lenient	xsmall	5.5%	94.9%	0.104	1,023
	base	6.2%	96.6%	0.116	921
Response, strict	xsmall	20.8%	52.1%	0.298	120
	base	24.7%	43.8%	0.316	85

Reading these numbers. Precision and recall of the HALLUCINATED class are reported; overall accuracy (86–92%) is not, because labelling everything clean would already score 96.5%.

- Strict sentence precision of 12.6% is about 3.6 times the 3.5% base rate, so the verifier carries real signal, but most of its flags are still false alarms.
- The base model is more cautious than xsmall (103 vs 188 flags, false alarms 90 vs 171) and catches slightly fewer (13 vs 17). With 59 positives, that difference is within noise; the models are best described as comparable.
- The lenient rule catches almost everything only by flagging over half of all sentences, so it is not a usable detector.
- **Why it is hard:** most RAGTruth hallucinations are "baseless info" (not in the source) rather than contradictions, and a pretrained NLI model classes those as neutral. Source documents are also long and seen in chunks.

> **Not comparable to published RAGTruth scores.** RAGTruth baselines are span-level; this project converts spans to sentence labels, so one short span marks a whole sentence.

7.3 Pending runs

```
Experiment B, fine-tune then evaluate (about 1–2 hours on CPU)
python -m training.train_verifier --allow-download --max-train 3000
python -m training.evaluate_verifier --model models\fever-verifier --split shared_task_dev

Experiment C with the fine-tuned model
$env:NLI_MODEL="models\fever-verifier"
python -m src.cli --verifier nli eval-ragtruth data\ragtruth\response.jsonl -n 300

Experiment D: dashboard → Ask & verify → Controlled Hallucination Test (8 reviewed examples)
```

8. Limitations

- **Sentences are not atomic facts.** Multi-assertion sentences are flagged, not split (unless LLM decomposition is switched on).
- **Lexical retrieval.** Wrong pages or passages usually give Insufficient Evidence, occasionally a spurious Contradicted on a related fact.
- **Model limits.** NLI models read at most 512 tokens per pair, and their scores are not calibrated probabilities.
- **Wikipedia is the reference.** Supported means "consistent with the retrieved Wikipedia text", which can itself be wrong or outdated. Today's Wikipedia also differs from FEVER's 2017 dump.
- **FEVER NEI evidence is heuristic** (nearest page), which affects that class's metrics.
- **RAGTruth conversion** is coarse (span → sentence), and 167 skipped sentences were excluded.
- **CPU-sized subsets.** 9,000 training claims and 3,000-claim evaluation sets were used to fit a CPU budget, not the full data.
- **Generator.** The generator is gpt-oss-20b on Groq rather than Llama 3.2 3B, and free-tier rate limits apply.

9. How to run it (Windows PowerShell)

```
cd $HOME\OneDrive\Desktop\hallucination-detection
pip install -r requirements.txt
Copy-Item .env.example .env          # then set GROQ_API_KEY in .env
python -m src.cli status             # generation, Wikipedia and verifier status

# NLI verifier (about 0.5 GB)
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements-nli.txt
python -m src.cli download-nli

streamlit run app.py                 # http://localhost:8501
```

Data layout: `data\fever\train.jsonl`, `data\fever\shared_task_dev.jsonl`, `data\fever\wiki-pages\`, `data\ragtruth\response.jsonl` and `source_info.jsonl`. Datasets, model weights and `.env` are excluded from git.

10. Code map and testing

Path	Responsibility
`app.py`	Streamlit dashboard
`src/config.py`, `src/schemas.py`	Settings from environment / `.env`; data types (Answer, Claim, Verdict, ...)
`src/llm/`	Generation backends behind one interface: `api_client.py` (Groq, OpenRouter, OpenAI, custom), `ollama_client.py`, `factory.py`
`src/claims/extractor.py`	spaCy sentence extraction, skip reasons, multi-assertion flags, optional LLM decomposition
`src/retrieval/`	Wikipedia search and text, passage ranking, cache; source-context retriever for RAGTruth
`src/verification/`	NLI verifier, LLM fallback, verifier selection
`src/pipeline.py`	Glues the stages together
`src/experiments/`	Controlled hallucination injection and batch mutations
`src/datasets/`, `src/dataset_eval.py`, `src/evaluation.py`	FEVER and RAGTruth loaders, evaluation runs, metrics and saved reports
`training/`	prepare_fever, train_verifier, evaluate_verifier
`tests/`	135 unit tests

Testing. 135 unit tests pass and the code is lint-clean (ruff). The tests cover every module with the LLM APIs, Wikipedia and the models replaced by test doubles, and real network access is blocked inside tests. A training smoke test trains a tiny random model end to end to check the code path; its numbers are meaningless and are never reported. The full dashboard flow was also run against mock services. The results in Section 7 come from real runs on Sreejay's computer.

References

- Thorne, J., Vlachos, A., Christodoulopoulos, C., Mittal, A. (2018). FEVER: a large-scale dataset for Fact Extraction and VERification. NAACL.
- Niu, C., Wu, Y., Zhu, J., et al. (2024). RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. ACL.
- He, P., Gao, J., Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.
- Honnibal, M., Montani, I., et al. spaCy: Industrial-strength Natural Language Processing in Python.